

Lab 4 VibeCoding – Chat History (Notion AI)

Student: SUKRUTH-ES | Assigned repo: SETAPESU26/32_lunarLander | Lab repo:
SUKRUTH-ES/SE_LAB4_32_Lunar_Lander
Date: 30 Sep 2026 | AI tool: Notion AI agent (chat session
<https://app.notion.com/chat?t=3eb4464eb7878085a9ba00a929851490>)

Prompt 1 – User

hey, i've got a lab assignment (handout attached). my assigned repo is https://github.com/SETAPESU26/32_lunarLander, it's a lunar lander game in pygame. the README has one bug to fix and three empty functions to fill in. make sure that u also read the handout i hv provided and follow it in future steps.

here's what i need u to do:

1. clone the repo and go through game.py and the README so you know what's what
2. fix the bug in touchdown(). right now you can slam down sideways way faster than MAX_SPEED_X and it still says perfect landing. it should check horizontal speed too, and if that's the reason for the crash, the message should say so
3. fill in the three functions:
 - ship_color(fuel_ratio): make the ship slowly turn red as fuel runs low (white when full, red when empty)
 - on_landing(score): add a fireworks/particle burst where the ship lands, bigger and more colorful on the x3 pad. a small beep would be nice too, but it shouldn't crash if there's no audio
 - bonus_life_threshold(): return 1500, and show a quick "BONUS LIFE!" on screen when you get one
4. please leave everything else as it is, and add a small comment like # TASK 1 next to each change so my instructor can find them
5. test that it actually runs and each part works
6. make two 10 second gameplay videos: a "before" one with the original code showing the sideways landing still counting as perfect, and an "after" one showing that same landing now crashing, the ship turning red on low fuel, the fireworks, and the bonus life. an auto-play script that saves frames and stitches them with ffmpeg is fine
7. put before.mp4, after.mp4 and the updated game.py in a folder called Lab-4, and give me all three files to download
8. then push the Lab-4 folder to my repo: https://github.com/SUKRUTH-ES/SE_LAB4_32_Lunar_Lander (my github username is SUKRUTH-ES). do NOT open a PR to SETAPESU26, the handout is strict about that. if you need me to sign in to github, ask me, and if you can't push for me, just give me the exact steps to upload it myself

make sure u push whatever is needed to the github link provided, then i also need u to export this chat history in a pdf/ doc and then push it to the same github link necessarily inside the lab4 folder where everything else is present, make sure that the read me file has everything necessary information

[Attachment: Lab_4_VibeCoding_handout.pdf – clone assigned repo, record 10 s before/after videos, fix bug, add README features one by one, push to personal repo under Lab-4 (videos, updated code, chat history as doc/pdf), do NOT raise a PR to SETAPESU26.]

Assistant – actions and responses

Step 1 – Read the handout and cloned https://github.com/SETAPESU26/32_lunarLander (clone only, no fork/PR). Read README.md and game.py (184 lines). Installed pygame 2.6.1.

Findings: touchdown() tests "pad and $\text{abs}(\text{self.vel.y}) \leq \text{MAX_SPEED_Y}$ and $\text{abs}(\text{angle}) \leq \text{MAX_ANGLE}$ " – MAX_SPEED_X is defined but never checked, which is the deliberate bug. ship_color(), on_landing() and bonus_life_threshold() are empty (return None).

Step 2 – Task 1 fix: added " $\text{abs}(\text{self.vel.x}) \leq \text{MAX_SPEED_X}$ " to the success condition, and a new crash reason "drifting sideways too fast" used when horizontal speed is the cause (vertical-speed crashes keep "too fast"). Tagged # TASK 1.

Step 3 – Features:

- Task 2 ship_color: linear blend from (230,230,240) at full fuel to (255,30,30) at empty.
- Task 3 on_landing: particle burst at the ship position – 60 gold/white sparks on the x1 pad, 180 multicolour faster sparks on the x3 pad, with gravity and fade-out; short synthesized beep (880 Hz / 1320 Hz) inside try/except so missing audio never crashes. Supporting hooks: FX state dict, Game.__init__ registers itself in FX, draw() calls draw_effects().
- Task 4 bonus_life_threshold: returns 1500; when a life is awarded, a "BONUS LIFE!" banner shows for ~1.5 s. All changes tagged # TASK n; nothing else modified.

Step 4 – Testing (headless, SDL dummy drivers):

- Original: sideways touchdown at Vx=60 → "Perfect landing! +400". Updated → "Crashed: drifting sideways too fast!".
- Gentle landings still succeed; particles 60 (x1) and 180 (x3).
- ship_color(1.0/0.5/0.0) = (230,230,240)/(242,130,135)/(255,30,30).
- Score 1600 → lives 3 → 4, banner active.
- Real main() loop ran for 4 s without error.

Step 5 – Videos: wrote record_demo.py (fixed seed, scripted keys, saves PNG frames, ffmpeg → H.264). Same script run on both versions: (1) sideways landing Vx=60; (2) low-fuel autopilot descent onto x3 pad with score preset to 1300; (3) next round where the bonus life triggers. before.mp4 shows "Perfect landing!" for the sideways slam; after.mp4 shows the crash, red hull, fireworks and "BONUS LIFE!". First take had the descent too slow, so timing was tuned and re-recorded.

Step 6 – Created Lab-4/ with game.py, before.mp4, after.mp4, README.md, changes.diff, record_demo.py and this chat history PDF, then pushed Lab-4 to https://github.com/SUKRUTH-ES/SE_LAB4_32_Lunar_Lander (no PR to SETAPESU26).

Code changes produced by the assistant (git diff of game.py)

```
diff --git a/game.py b/game.py
index 8ed5edf..b51a3be 100644
--- a/game.py
+++ b/game.py
@@ -9,19 +9,82 @@ MAX_SPEED_X, MAX_SPEED_Y, MAX_ANGLE = 25.0, 40.0, 0.25
 FOOT = 12

+# TASK 3/4: shared state for visual effects (particles, beep, bonus-life banner)
+FX = {"game": None, "particles": [], "bonus_timer": 0, "beep": None}
+
+
+def ship_color(fuel_ratio):
+    """Return an (r, g, b) hull colour for the given fuel ratio (1.0 = full), or None for the default."""
+    pass
+    # TASK 2: blend smoothly from white-ish (full tank) to red (empty tank)
+    t = max(0.0, min(1.0, fuel_ratio))
+    full, empty = (230, 230, 240), (255, 30, 30)
+    return tuple(int(e + (f - e) * t) for f, e in zip(full, empty))
+
+
+def play_beep(freq, duration=0.15):
+    # TASK 3: short synthesized beep; silently skipped if there is no audio device
+    try:
+        if not pygame.mixer.get_init():
+            pygame.mixer.init()
+        rate, _, channels = pygame.mixer.get_init()
+        from array import array
+        samples = array("h")
+        n = int(rate * duration)
+        for i in range(n):
+            v = int(9000 * math.sin(math.tau * freq * i / rate) * (1 - i / n))
+            samples.extend([v] * channels)
+        FX["beep"] = pygame.mixer.Sound(buffer=samples.tobytes())
+        FX["beep"].play()
+    except Exception:
+        pass
+
+def on_landing(score):
```

```

        """Called after a successful landing with the points just earned; add fireworks or bonuses here."""
-    pass
+    # TASK 3: fireworks burst at the landing spot, bigger and more colourful on the x3 pad
+    game = FX["game"]
+    if game is None:
+        return
+    pad = game.pad_under()
+    mult = pad[3] if pad else 1
+    count = 60 if mult == 1 else 180
+    if mult == 1:
+        palette = [(255, 220, 90), (255, 255, 255)]
+    else:
+        palette = [(255, 80, 80), (255, 200, 60), (90, 230, 120), (80, 170, 255), (220, 100, 255), (255, 255, 255)]
+    speed = 120 if mult == 1 else 220
+    for _ in range(count):
+        a = random.uniform(math.pi, math.tau) # upward half-circle
+        v = random.uniform(0.3, 1.0) * speed
+        FX["particles"].append([game.pos.x, game.pos.y, math.cos(a) * v, math.sin(a) * v,
+                                random.uniform(0.8, 1.6), random.choice(palette)])
+    play_beep(880 if mult == 1 else 1320)

def bonus_life_threshold():
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""
-    pass
+    # TASK 4: one extra life every 1500 points
+    return 1500
+
+
+def draw_effects(screen, font):
+    # TASK 3/4: advance + draw fireworks particles and the "BONUS LIFE!" banner (called from draw)
+    dt = 1 / 60
+    alive = []
+    for p in FX["particles"]:
+        p[0] += p[2] * dt
+        p[1] += p[3] * dt
+        p[3] += 90 * dt # a little gravity so sparks arc down
+        p[4] -= dt
+        if p[4] > 0:
+            alive.append(p)
+        pygame.draw.circle(screen, p[5], (int(p[0]), int(p[1])), 3 if p[4] > 0.5 else 2)
+    FX["particles"] = alive
+    if FX["bonus_timer"] > 0:
+        FX["bonus_timer"] -= 1
+        label = font.render("BONUS LIFE!", True, (255, 215, 0))
+        label = pygame.transform.scale_by(label, 2)
+        screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 3 + 50)))

def make_terrain():
@@ -51,6 +114,7 @@ def wrap_angle(angle):
    class Game:
        def __init__(self):
            self.font = pygame.font.Font(None, 26)
+            FX["game"] = self # TASK 3: let on_landing() find the ship position
            self.reset()

        def reset(self):
@@ -74,7 +138,7 @@ class Game:
        def touchdown(self):
            pad = self.pad_under()
            angle = wrap_angle(self.angle)
-            if pad and abs(self.vel.y) <= MAX_SPEED_Y and abs(angle) <= MAX_ANGLE:
+            if pad and abs(self.vel.x) <= MAX_SPEED_X and abs(self.vel.y) <= MAX_SPEED_Y and abs(angle) <= MAX_ANGLE: # TASK
                earned = int((100 + self.fuel) * pad[3])
                self.score += earned
                self.state, self.message = "landed", f"Perfect landing! +{earned} (Space = next level)"
@@ -86,8 +150,10 @@ class Game:
            reason = "missed the pad"
            elif abs(angle) > MAX_ANGLE:
                reason = "bad angle"
-            else:
+            elif abs(self.vel.y) > MAX_SPEED_Y:
                reason = "too fast"
+            else:
                reason = "drifting sideways too fast" # TASK 1: horizontal speed was the cause

```

```

        self.message = f"Crashed: {reason}! " + ("Space = retry" if self.lives > 0 else "Game over - R = restart")

    def update(self, dt, keys):
@@ -97,6 +163,7 @@ class Game:
        if threshold and self.score // threshold > self.bonus_awarded:
            self.bonus_awarded = self.score // threshold
            self.lives += 1
+            FX["bonus_timer"] = 90 # TASK 4: show "BONUS LIFE!" for ~1.5 s
        self.angle += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * ROTATE_SPEED * dt
        gravity = pygame.Vector2(0, 16 + 2 * self.level)
        self.thrusting = bool(keys[pygame.K_UP]) and self.fuel > 0
@@ -152,6 +219,7 @@ class Game:
        if self.message:
            label = self.font.render(self.message, True, (255, 255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 3)))
+            draw_effects(screen, self.font) # TASK 3/4: fireworks + bonus banner

def main():

```